OJT TRACKER

SQLite + Python Integration Guide

A Step-by-Step Beginner's Guide

Learn Python • SQLite • User Login • Build .EXE

What you will learn in this guide:

How to set up a SQLite database • How databases store data • How to build a login system
How to connect SQLite to your app • How to build your project into an .EXE file

What is SQLite and Why Use It?

Before writing any code, it helps to understand what SQLite actually is and why it is better for your OJT Tracker than saving data to a JSON file.

SQLite vs. JSON — Side by Side

Feature	JSON File (old way)	SQLite (new way)
Multiple users	Cannot separate users	Each user has their own data
Passwords	Not supported	Stored as secure hashes
Speed	Slow for large data	Fast with indexes
Data safety	Easy to corrupt	Built-in transaction safety
Viewing data	Open a text file	Use VS Code SQLite extension
Storage format	Plain text .json file	Binary .db file

> **How to think about a database**
> A SQLite database is like an Excel workbook. Each "table" is a sheet with rows and columns. Your OJT Tracker will have 4 tables: users, tracker_state, daily_notes, and daily_status. Every user gets their own rows — no data mixing.

Setup — Before You Write Any Code

Make sure you have these tools installed. Open your terminal (cmd or PowerShell on Windows) and run each command to check.

STEP 1

Check Your Python Version

Python 3.8 or higher is required

Open your terminal and type:

```
python --version
```

You should see something like: Python 3.11.4. If you see an error, download Python from python.org.

Good to know

Python already includes sqlite3 — you do NOT need to install anything extra for the database itself. The SQLite extension in VS Code (by Alex) is just for visually viewing the .db file.

STEP 2

Install Required Packages

These are the Python libraries your project needs

Open your terminal in your project folder (where ojt_tracker2.py lives) and run:

```
pip install customtkinter
pip install awesometkinter
pip install tkcalendar
pip install pyinstaller
```

What is pip?

pip is Python's package manager — it downloads and installs libraries. Think of it like an app store for Python code. You only need to run these once.

STEP

3

Set Up Your Project Folder Structure

Keep files organized from the start

Your project folder should look like this when you are done:

```
ojt_tracker/
  ├── ojt_tracker2.py     ← your main app file (already exists)
  ├── db_manager.py       ← new file we will create (Step 4)
  ├── ojt_tracker.db      ← created automatically when app runs
  └── ojt_tracker.lock    ← created automatically
```

Important

The .db file is created automatically the first time you run the app. You do NOT need to create it manually. Just having db_manager.py is enough.

Step 4 — Create the Database Manager File

This is the most important file you will create. The db_manager.py file is a helper module — it handles all communication between your app and the SQLite database. Think of it as the "middleman".

What is a module?

A module is just a separate Python file that you can import into another file. By keeping all database code in db_manager.py, your main app file stays clean and readable.

STEP 4

Create db_manager.py

Create this file in the same folder as ojt_tracker2.py

Part A — Imports and Connection

At the top of db_manager.py, add these lines:

```
import sqlite3
import hashlib
import os
from datetime import datetime

DB_FILE = "ojt_tracker.db"  # This is the name of your database file

def get_connection():
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row  # lets you access columns by name
    return conn
```

What does this do?

sqlite3.connect() opens (or creates) the database file. conn.row_factory = sqlite3.Row lets you access results like a dictionary — for example row["username"] instead of row[0].

Part B — Create the Tables

Tables are where data lives. Add this function after get_connection():

```python
def init_db():
    """Create all tables if they don't exist yet"""
    conn = get_connection()
    cursor = conn.cursor()  # cursor is what actually runs SQL commands

    # TABLE 1: stores user accounts
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id        INTEGER PRIMARY KEY AUTOINCREMENT,
            username  TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL,
            full_name TEXT,
            created_at TEXT DEFAULT (datetime('now')),
            total_hours_required REAL DEFAULT 300.0
        )
    """)

    # TABLE 2: stores each user's OJT progress
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS tracker_state (
            user_id          INTEGER PRIMARY KEY,
            remaining_seconds REAL NOT NULL,
            is_clocked_in    INTEGER DEFAULT 0,
            is_on_break      INTEGER DEFAULT 0,
            last_closed      TEXT,
            FOREIGN KEY (user_id) REFERENCES users(id)
        )
    """)

    # TABLE 3: stores notes per day per user
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS daily_notes (
            id        INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id   INTEGER NOT NULL,
            note_date TEXT NOT NULL,
            note_text TEXT,
            UNIQUE(user_id, note_date),
            FOREIGN KEY (user_id) REFERENCES users(id)
        )
    """)

    # TABLE 4: stores attendance status per day per user
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS daily_status (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id     INTEGER NOT NULL,
            status_date TEXT NOT NULL,
            status      TEXT,
            UNIQUE(user_id, status_date),
            FOREIGN KEY (user_id) REFERENCES users(id)
        )
    """)

    conn.commit()  # save all changes
```

```
    conn.close()   # always close when done
```

Key SQL concepts used here

CREATE TABLE IF NOT EXISTS — only creates the table if it does not already exist (safe to call every startup). INTEGER PRIMARY KEY AUTOINCREMENT — auto-numbers each row (1, 2, 3...). FOREIGN KEY — links tables together, so every note/status is tied to a real user.

Part C — Password Hashing

Never store passwords as plain text. Use hashing — a one-way scramble that cannot be reversed:

```
def hash_password(password):
    """Convert a plain password into a secure hash"""
    return hashlib.sha256(password.encode()).hexdigest()

# Example:
# hash_password("mypassword123") returns:
# "ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f"
# This is what gets stored — not the real password
```

Part D — Register and Login Functions

```
def register_user(username, password, full_name=""):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO users (username, password_hash, full_name) VALUES (?, ?, ?)",
            (username.strip(), hash_password(password), full_name.strip())
        )
        user_id = cursor.lastrowid  # get the ID of the new user

        # Also create an empty tracker_state row for this user
        cursor.execute(
            "INSERT INTO tracker_state (user_id, remaining_seconds) VALUES (?, ?)",
            (user_id, 300 * 3600)  # 300 hours in seconds
        )
        conn.commit()
        return True, "Account created successfully!"
    except sqlite3.IntegrityError:
        return False, "Username already exists. Please choose another."
    except Exception as e:
```

```
        return False, f"Error: {e}"
    finally:
        conn.close()  # always runs, even if there was an error


def login_user(username, password):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT * FROM users WHERE username = ? AND password_hash = ?",
            (username.strip(), hash_password(password))
        )
        user = cursor.fetchone()  # get the first matching row
        if user:
            return True, dict(user)  # convert Row to dictionary
        return False, None
    finally:
        conn.close()
```

Why use ? placeholders?

The ? in SQL queries are called parameterized queries. They prevent SQL injection — a common security attack where someone types SQL code into a text field to manipulate your database. Always use ? instead of string formatting in SQL.

Part E — Save and Load Tracker State

```
def save_tracker_state(user_id, remaining_seconds, is_clocked_in, is_on_break,
include_close_time=False):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        last_closed = datetime.now().strftime("%B %d, %Y at %I:%M:%S %p") if
include_close_time else None

        if last_closed:
            cursor.execute("""
                UPDATE tracker_state
                SET remaining_seconds=?, is_clocked_in=?, is_on_break=?, last_closed=?
                WHERE user_id=?
            """, (remaining_seconds, int(is_clocked_in), int(is_on_break), last_closed,
user_id))
        else:
            cursor.execute("""
                UPDATE tracker_state
                SET remaining_seconds=?, is_clocked_in=?, is_on_break=?
                WHERE user_id=?
            """, (remaining_seconds, int(is_clocked_in), int(is_on_break), user_id))
```

```
        conn.commit()
    finally:
        conn.close()


def load_tracker_state(user_id):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM tracker_state WHERE user_id = ?", (user_id,))
        row = cursor.fetchone()
        return dict(row) if row else None
    finally:
        conn.close()
```

Part F — Notes and Status Functions

```
def save_note(user_id, note_date, note_text):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO daily_notes (user_id, note_date, note_text) VALUES (?, ?, ?)
            ON CONFLICT(user_id, note_date) DO UPDATE SET note_text=excluded.note_text
        """, (user_id, note_date, note_text))
        conn.commit()
    finally:
        conn.close()

def delete_note(user_id, note_date):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("DELETE FROM daily_notes WHERE user_id=? AND note_date=?",
(user_id, note_date))
        conn.commit()
    finally:
        conn.close()

def load_notes(user_id):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT note_date, note_text FROM daily_notes WHERE user_id=?",
(user_id,))
        return {row['note_date']: row['note_text'] for row in cursor.fetchall()}
    finally:
        conn.close()

def save_status(user_id, status_date, status):
```

```
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO daily_status (user_id, status_date, status) VALUES (?, ?, ?)
            ON CONFLICT(user_id, status_date) DO UPDATE SET status=excluded.status
        """, (user_id, status_date, status))
        conn.commit()
    finally:
        conn.close()

def delete_status(user_id, status_date):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("DELETE FROM daily_status WHERE user_id=? AND status_date=?",
(user_id, status_date))
        conn.commit()
    finally:
        conn.close()

def load_statuses(user_id):
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT status_date, status FROM daily_status WHERE user_id=?",
(user_id,))
        return {row['status_date']: row['status'] for row in cursor.fetchall()}
    finally:
        conn.close()
```

ON CONFLICT ... DO UPDATE

This is called an "upsert" — it means: insert a new row, but if a row with that user_id + date already exists, just update it instead. This is perfect for saving notes and statuses because you do not need separate insert and update logic.

Step 5 — Add the Login Window to ojt_tracker2.py

Now you need to update your main app file. The login window will appear first, before the main tracker window. Only after a successful login will the main app open.

STEP 5

Add These Lines at the Top of ojt_tracker2.py

Import db_manager and initialize the database

Find the top of your ojt_tracker2.py file — right after all the existing import statements — and add:

```
import db_manager          # import our new database helper

db_manager.init_db()       # create tables if they don't exist yet
current_user = None        # will store the logged-in user's data
```

STEP 6

Add the Login Window Function

Paste this function before app = ctk.CTk()

```
def show_login_window():
    global current_user

    login_win = ctk.CTk()
    login_win.title("OJT Tracker - Login")
    login_win.geometry("360x420")
    login_win.resizable(False, False)

    # Center the window on screen
    sw = login_win.winfo_screenwidth()
    sh = login_win.winfo_screenheight()
    login_win.geometry(f"360x420+{int(sw/2-180)}+{int(sh/2-210)}")

    mode_var = ctk.StringVar(value="login")  # tracks if we are in login or register mode

    # Title labels
    ctk.CTkLabel(login_win, text="OJT Tracker",
                 font=ctk.CTkFont(size=28, weight="bold")).pack(pady=(30, 5))
    subtitle = ctk.CTkLabel(login_win, text="Sign in to your account",
                            font=ctk.CTkFont(size=13), text_color="gray")
```

```
    subtitle.pack(pady=(0, 20))

    # Input fields
    fullname_entry = ctk.CTkEntry(login_win, width=260, placeholder_text="Full Name")
    username_entry = ctk.CTkEntry(login_win, width=260, placeholder_text="Username")
    username_entry.pack(pady=5)
    password_entry = ctk.CTkEntry(login_win, width=260, placeholder_text="Password",
show="*")
    password_entry.pack(pady=5)

    # Message label for errors or success
    message_label = ctk.CTkLabel(login_win, text="",
                                 font=ctk.CTkFont(size=12), text_color="#f44336")
    message_label.pack(pady=5)

    def do_login():
        global current_user
        success, user_data = db_manager.login_user(username_entry.get(),
password_entry.get())
        if success:
            current_user = user_data
            login_win.destroy()  # close login window, main app opens
        else:
            message_label.configure(text="Invalid username or password.",
text_color="#f44336")

    def do_register():
        success, msg = db_manager.register_user(
            username_entry.get(), password_entry.get(), fullname_entry.get()
        )
        if success:
            message_label.configure(text=msg, text_color="#4CAF50")
            toggle_mode()  # go back to login mode after registering
        else:
            message_label.configure(text=msg, text_color="#f44336")

    action_btn = ctk.CTkButton(login_win, width=260, text="Login", command=do_login,
                               fg_color="#1f6aa5", hover_color="#174F7C")
    action_btn.pack(pady=10)

    def toggle_mode():
        if mode_var.get() == "login":
            mode_var.set("register")
            subtitle.configure(text="Create a new account")
            fullname_entry.pack(before=username_entry, pady=5)
            action_btn.configure(text="Register", command=do_register)
            toggle_btn.configure(text="Already have an account? Login")
        else:
            mode_var.set("login")
            subtitle.configure(text="Sign in to your account")
            fullname_entry.pack_forget()
            action_btn.configure(text="Login", command=do_login)
            toggle_btn.configure(text="Don't have an account? Register")
        message_label.configure(text="")
```

```
    toggle_btn = ctk.CTkButton(
        login_win, text="Don't have an account? Register",
        fg_color="transparent", hover_color="#333",
        command=toggle_mode, font=ctk.CTkFont(size=12)
    )
    toggle_btn.pack(pady=5)

    # Allow pressing Enter to submit
    login_win.bind("<Return>", lambda e: do_login() if mode_var.get() == "login" else
do_register())
    login_win.mainloop()

    if current_user is None:
        import sys
        sys.exit(0)  # user closed window without logging in
```

STEP

7

Call the Login Window

Add this line right before app = ctk.CTk()

```
# Add this line right before:  app = ctk.CTk()
show_login_window()

# Then your existing line:
app = ctk.CTk()
app.title("OJT Tracker")
# ... rest of your code
```

Step 8 — Replace save_data and load_data

Your existing save_data() and load_data() functions write to a JSON file. You need to replace them with versions that use the database. Find these two functions in ojt_tracker2.py and replace them completely.

STEP
8

Replace save_data()

Find the old function and replace it entirely

Find this in your file:

```
def save_data(include_close_time=False):
    try:
        data = {
            ...
        }
        ...
```

Replace it with:

```
def save_data(include_close_time=False):
    """Save tracker state to SQLite database"""
    db_manager.save_tracker_state(
        current_user["id"],
        remaining_seconds,
        is_clocked_in,
        is_on_break,
        include_close_time
    )
```

STEP
9

Replace load_data()

Find the old function and replace it entirely

```
def load_data():
    """Load tracker state from SQLite database"""
    global remaining_seconds, is_clocked_in, is_on_break
    global total_hours_required, daily_notes, daily_status
```

```
    state = db_manager.load_tracker_state(current_user["id"])
    if state:
        remaining_seconds = state["remaining_seconds"]
        is_clocked_in = bool(state["is_clocked_in"])
        is_on_break   = bool(state["is_on_break"])
        last_closed   = state.get("last_closed")
        if last_closed:
            last_closed_label.configure(text=f"Last closed: {last_closed}")
        else:
            last_closed_label.configure(text="Never closed")

    total_hours_required = current_user.get("total_hours_required", 300)
    daily_notes  = db_manager.load_notes(current_user["id"])
    daily_status = db_manager.load_statuses(current_user["id"])
    update_ui_from_data()
```

STEP

10

Update Calendar Note and Status Saves

Small updates inside the calendar functions

Inside your open_calendar() function, find the save_note() inner function and update it:

```
def save_note():
    note_content = note_text.get("0.0", "end").strip()
    daily_notes[date] = note_content
    db_manager.save_note(current_user["id"], date, note_content)  # save to DB
    note_window.destroy()
```

Find the delete_note() inner function and update it:

```
def delete_note():
    result = messagebox.askyesno("Delete Note", f"Delete note for {date}?")
    if result:
        if date in daily_notes:
            del daily_notes[date]
        db_manager.delete_note(current_user["id"], date)  # delete from DB
        note_window.destroy()
```

Find the set_status() function and update it:

```
def set_status(status, color):
    selected_date_str = cal.get_date()
```

```
    selected_date_obj = str_to_date(selected_date_str)
    daily_status[selected_date_str] = status
    for event_id in cal.get_calevents(selected_date_obj):
        cal.calevent_remove(event_id)
    cal.calevent_create(selected_date_obj, status, status)
    cal.tag_config(status, background=color, foreground="white")
    db_manager.save_status(current_user["id"], selected_date_str, status)  # save to DB
```

Find the clear_status() function and update it:

```
def clear_status():
    selected_date_str = cal.get_date()
    selected_date_obj = str_to_date(selected_date_str)
    if selected_date_str in daily_status:
        del daily_status[selected_date_str]
    for event_id in cal.get_calevents(selected_date_obj):
        cal.calevent_remove(event_id)
    db_manager.delete_status(current_user["id"], selected_date_str)  # delete from DB
```

Step 11 — View Your Database in VS Code

Now that your app saves to SQLite, you can inspect the data using the SQLite extension by Alex that you already have installed.

STEP
11

Open the Database in VS Code

See your tables and data visually

1. Run your app and create an account / log in — this creates the .db file
2. In VS Code, find the file ojt_tracker.db in your project folder
3. Right-click the .db file and choose Open Database
4. In the bottom-left of VS Code, you will see SQLITE EXPLORER
5. Expand it to see your 4 tables: users, tracker_state, daily_notes, daily_status
6. Right-click any table and choose Show Table to see all rows

What you will see

The users table will show your registered accounts. Passwords appear as a long string of letters and numbers — that is the hash, not the real password. The tracker_state table shows remaining_seconds for each user.

Running SQL Queries Manually

You can also type SQL directly. In VS Code with the SQLite extension, open the command palette (Ctrl+Shift+P) and type "SQLite: Open Database", then "SQLite: New Query" to run commands like:

```
-- See all users
SELECT id, username, full_name, created_at FROM users;

-- See tracker state for user ID 1
SELECT remaining_seconds / 3600.0 AS hours_left FROM tracker_state WHERE user_id = 1;

-- See all notes
SELECT u.username, n.note_date, n.note_text
FROM daily_notes n
JOIN users u ON u.id = n.user_id;
```

Step 12 — Build the .EXE File

Once your app is working correctly, you can package it into a single .exe file using PyInstaller. This lets you share the app with others who do not have Python installed.

STEP 12

Test Your App First

Make sure everything works before building

Run your app normally and test all features:

```
python ojt_tracker2.py
```

Important: Fix all bugs before building!
Every time you change your Python code after building, you must rebuild the .exe. It is much faster to debug using python ojt_tracker2.py directly.

STEP 13

Run the PyInstaller Command

Open your terminal in the project folder

Basic build — creates a single .exe file:

```
pyinstaller --onefile --windowed --name "OJT Tracker" ojt_tracker2.py
```

What Each Flag Means

Flag	What it does
`--onefile`	Packs everything into a single .exe (easier to share)
`--windowed`	Hides the black console/terminal window when running
`--name "OJT Tracker"`	Sets the name of the output .exe file

`--icon=icon.ico`	Optional: adds a custom icon to the .exe

If CustomTkinter or AwesomeTkinter Have Errors

Some libraries need to be collected explicitly. Use this extended command if you get import errors:

```
pyinstaller --onefile --windowed \
  --collect-all customtkinter \
  --collect-all awesometkinter \
  --name "OJT Tracker" \
  ojt_tracker2.py
```

Where is the .EXE?

```
ojt_tracker/
  ├── dist/
  │   └── OJT Tracker.exe    ← your finished executable is here
  ├── build/                  ← temporary build files (can delete)
  ├── OJT Tracker.spec        ← build configuration (keep this)
  ├── ojt_tracker2.py
  └── db_manager.py
```

> **About the .db file with the .exe**
>
> The ojt_tracker.db database file is NOT inside the .exe — it is created in the same folder as the .exe when the app first runs. This is intentional: it means your data is saved even when you update the .exe to a new version.

Quick Reference — SQL Commands Cheat Sheet

Here are the most common SQL operations you will use. Study these patterns and you will be able to write your own database functions.

The 4 Basic Operations (CRUD)

```
-- CREATE (insert a new row)
INSERT INTO users (username, password_hash) VALUES ('john', 'abc123hash');

-- READ (get rows)
SELECT * FROM users;                              -- get all users
SELECT username FROM users WHERE id = 1;          -- get one user by ID
SELECT * FROM users WHERE username = 'john';      -- get by username

-- UPDATE (change existing data)
UPDATE tracker_state SET remaining_seconds = 1080000 WHERE user_id = 1;

-- DELETE (remove rows)
DELETE FROM daily_notes WHERE user_id = 1 AND note_date = '01/15/25';
```

Python sqlite3 Patterns

```
import sqlite3

# 1. Open a connection
conn = sqlite3.connect('myapp.db')
conn.row_factory = sqlite3.Row   # enables row['column_name'] access

# 2. Create a cursor
cursor = conn.cursor()

# 3. Execute a query (always use ? for values)
cursor.execute('SELECT * FROM users WHERE username = ?', ('john',))

# 4. Get results
row  = cursor.fetchone()    # get one row
rows = cursor.fetchall()    # get all rows

# 5. Save changes (required for INSERT/UPDATE/DELETE)
conn.commit()
```

```
# 6. Always close the connection
conn.close()
```

Common Mistakes and Fixes

Mistake	Fix
Forgetting conn.commit() after INSERT/UPDATE/DELETE	Add conn.commit() before conn.close()
Forgetting conn.close() — causes file locking issues	Use try/finally to always close
Putting values directly in SQL string: f'...{val}...'	Always use ? placeholders
fetchone() returns None but you try to access it	Always check: if row: before using row
OperationalError: no such table	Make sure init_db() is called at startup

Final Checklist

db_manager.py created with all functions • import db_manager added to ojt_tracker2.py • db_manager.init_db() called at startup • show_login_window() called before app = ctk.CTk() • save_data() and load_data() updated • Calendar note/status saves updated • App tested with python ojt_tracker2.py • EXE built with pyinstaller